



**CS411**  
**SOFTWARE ARCHITECTURE DESIGN**  
**2025-2026 FALL**  
**PROJECT # 2**

**Ayşegül Yıldız- 22002591**  
**Musa Yiğit Yayla-22003108**

# **1. Architecture of the CRM System**

## **1.1. Stakeholders and Their Expectations & Contributions**

### **1.1.1. Sales staff**

Sales staff want a clean interface to track leads, update deals and get reminders for follow-ups. They also expect accurate sales predictions so they can plan their deals better. Their main contribution is providing high quality and accurate data about their potential customers, deals and their conversations with customers. This data is essential for sales prediction, reporting and automation features.

### **1.1.2. Sales managers**

Sales managers expect the CRM to make the sale process visible. They want dashboards that show deal progress, sales staff performance and revenue predictions. They expect forecasting tools to be reliable and current. They contribute by defining sales processes, setting sales stages, deciding on the scoring for sales staff and ensuring their teams use the system consistently.

### **1.1.3. Marketing team**

Marketing staff expects the CRM to give them clean data for customer grouping for personalized campaigns. They need automation tools to run email campaigns, track their social media activity. They also expect analytics to measure how much revenue marketing activities generate compared to their cost and understand which activities bring profit. Their contribution is organizing and managing campaigns, managing the rules used to group customers and improving the quality of marketing data such as audience lists.

### **1.1.4. Customers**

Customers expect faster, personalized service. They want support and product recommendations. Their indirect contribution is providing feedback through surveys, interactions and behavior data improving the CRM's intelligence.

### **1.1.5. Customer support agents**

Support agents expect the CRM to help them resolve issues quickly. They need fast access to customer history and product details. They expect the system to send each customer request to the most suitable agent based on skills and availability. Their main contribution is helping customers, logging customer interaction and updating ticket statuses. That information helps measure support performance, identify customers that may leave and improve service quality.

### **1.1.6. Customer support team managers**

These managers expect the CRM to ensure high service standards. They want dashboards showing how many customer requests are waiting, whether the team is late in helping anyone, how well each agent is performing, and how satisfied customers are. The system must be able to pass a problem to someone with more authority when needed. Their contribution is designing support processes, defining priority rules and ensuring the knowledge base is regularly updated.

### **1.1.7. Data scientists**

Data teams expect the CRM to give structured, high quality, reliable customer data. They need data about how the customers behave so they can create

predictions like which customers might leave and which customers are most valuable. Their contribution is creating the models that shape insights. They also refine data pipelines and help keep data consistent across modules.

#### **1.1.8. IT team**

The IT team expects the CRM to be secure, stable and easy to manage. They take care of the company's internal computers, networks and access systems, so they want the CRM to fit well with these tools. Their contribution is creating user accounts, managing permissions, keeping the internal network running and helping the CRM connect properly with the company's existing systems.

#### **1.1.9. DevOps**

DevOps engineers expect the CRM to run in the cloud without a problem and to be easy to update. They want the system to handle high traffic, recover quickly if something goes wrong and be simple to monitor. Their contribution is setting up the cloud environment, automating deployments, keeping the system online during busy times and making sure the CRM works fast and without failure.

#### **1.1.10. System Administrators**

System admins expect the CRM to be easy to set up without needing technical changes or writing code. They focus on the daily setup such as creating fields, updating forms, adjusting workflows and making sure each team sees the right data. Their contribution is keeping the CRM organized and working for all departments by managing settings inside the system.

#### **1.1.11. C level leaders**

Executives expect the CRM to act as a source of truth for all customer related decisions. They want dashboards showing revenue, how well the marketing is performing and customer value. They also expect the system to support strategic goals. Their contribution is approving budgets, defining business priorities and making sure the whole company uses the CRM system properly.

#### **1.1.12. External partners**

External partners and resellers expect the system to calculate their commissions correctly. They also expect the approval and payment process to be clear and error free. Their contribution is sending deals or referrals to the company which the system then uses to calculate commissions.

#### **1.1.13. Legal team**

Compliance teams expect the CRM to meet data protection laws and internal standards. They want features like access auditing, consent tracking, data retention rules and the ability to export or delete customer data when required. Their contribution is defining privacy policies, identifying legal risks and ensuring the CRM's data handling practices follow regulations.

## **1.2. Main Requirements and Quality Attributes**

### **1.2.1. Functional requirements**

#### **1.2.1.1. Sales automation requirements**

The sales automation module should help the company manage its sales process. It should allow sales teams to capture new leads, score and assign

them and also track opportunities throughout the pipeline. The system should give sales managers revenue predictions by using data, sales performance. Furthermore, the CRM must handle the calculation, approval and payment of commissions for both internal staff and external partners.

#### **1.2.1.2. Customer service and support requirements**

The customer service and support module must help the company respond to customer problems quickly and effectively. It needs to allow agents to log, prioritize, assign and track all customer issues until they are solved. The system should meet promised response and resolution times. The CRM must have a searchable knowledge base containing helpful articles, FAQs and guides. Both customers and staff should be able to use them to solve problems themselves. The system should accept service requests from different channels and automatically route each request to the most suitable agent based on skills, availability and customer priority.

#### **1.2.1.3. Marketing automation requirements**

The marketing automation module should help the company design, run and measure marketing campaigns. The system needs to group customers based on their history and demographic information so that marketing teams can send personalized messages. It must support automated emails, social media posts and campaigns that are triggered by customer actions. The CRM should provide analytics that show how well each campaign performs, how many possible customers turn into real customers, and how much revenue is generated as a result of marketing activities.

#### **1.2.1.4. Data Intelligence and customer support requirements**

The data intelligence and customer insight module must give the company a complete view of customers. It should calculate the long term financial value of every customer and help the company identify which customers may stop doing business with them. This allows the service team to take action before the customer becomes inactive. The system also needs to offer real time suggestions to sales and service teams, show them the most effective action or offer to present during customer interactions.

### **1.2.2. Nonfunctional requirements**

#### **1.2.2.1. Security and data privacy**

Security is essential because the system must protect customer data through secure logins, encrypted communication and controlled access. Only authorized users should see or change sensitive information. The CRM must follow data protection rules allowing customers to view or update their personal information and keeping records of who accesses or modifies data.

#### **1.2.2.2. Performance**

The CRM must have high performance so users can complete their work without delays. Pages should load quickly even when many people use the

system at the same time. Customer support requests must be routed in real time and calculations should be performed efficiently.

#### **1.2.2.3. Reliability**

The system should be available with minimal downtime and should not lose customer or sales data. Regular backups and recovery procedures should exist, the CRM should remain stable. Teams should be able to trust that the CRM will be working whenever they need it.

#### **1.2.2.4. Scalability**

As the business grows, the CRM must be able to grow with it. The system should handle increasing amounts of data, customer interactions and users without performance problems.

#### **1.2.2.5. Usability**

The CRM should have a simple interface that all teams can use easily. Users should be able to find information, update data without confusion.

#### **1.2.2.6. Maintainability**

The system must be easy to update and maintain without disrupting daily operations. Everything should be adjustable as business needs change.

#### **1.2.2.7. Interoperability**

The CRM should integrate well with the company's other systems such as email tools, internal software. It must offer reliable APIs so information can move between systems safely.

### **1.3. High-Level Architecture**

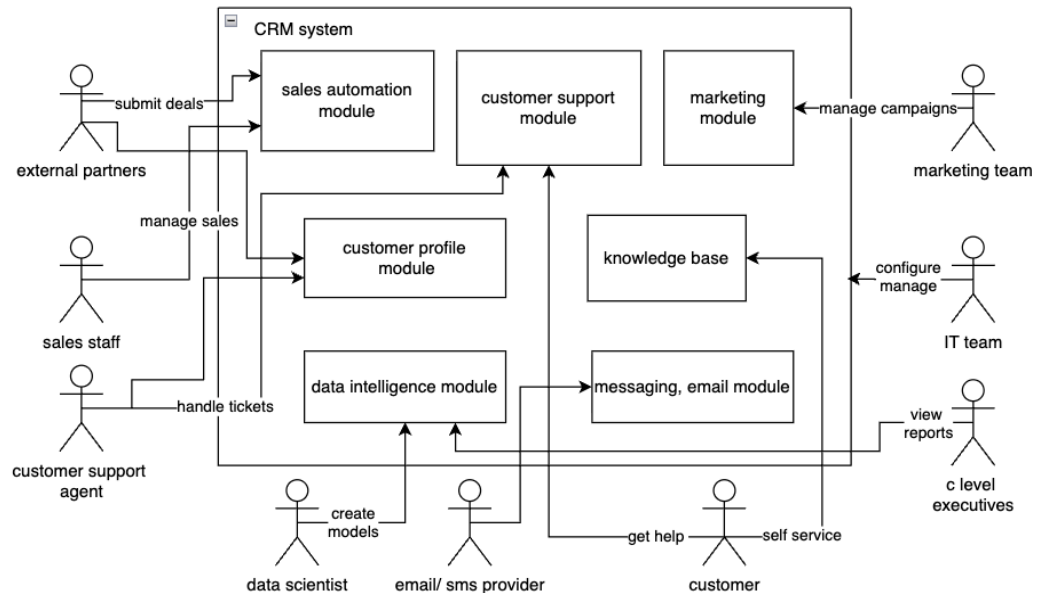
The high level architecture of the CRM system is built around the four major business modules defined as sales automation, customer service and support, marketing automation and data intelligence& customer insight. To support these modules, the CRM is organized as a unified platform with shared data and shared technical services. At the center of the system, a common customer data layer ensures that every module uses the same, current customer information, enabling the CRM to function as the company's single source of truth. Each module interacts with this shared data layer while focusing on its specific responsibilities. Overall, the architecture provides a modular foundation where each major business capability is clearly separated but still able to collaborate through shared customer data and common platform services.

#### **1.3.1. Context view**

The context view shows the CRM system as a single combined platform and shows how different users and external systems interact with it. In the diagram the CRM is at the center and displays the main internal modules showing the roles of the system. Around the CRM, the main stakeholders are shown. Each actor is connected to the modules they use the most, helping us understand interactions. This view represents the overall boundaries of the product.

The purpose of drawing the context view in this way is to give a high-level picture of how the CRM fits into its environment before we go deeper into technical details. Not every possible user action is shown because listing all

detailed interactions would make the diagram too crowded. For example, sales staff also update customer information. These interactions exist, but they are not all shown because the goal here is to show relationships at a high level. Overall, the context view provides a simple but complete understanding of the CRM's position in its environment. It lays the foundation for the more detailed architectural views that we will focus on next.



**Figure 1- Context view of CRM system**

### 1.3.2. Module Styles

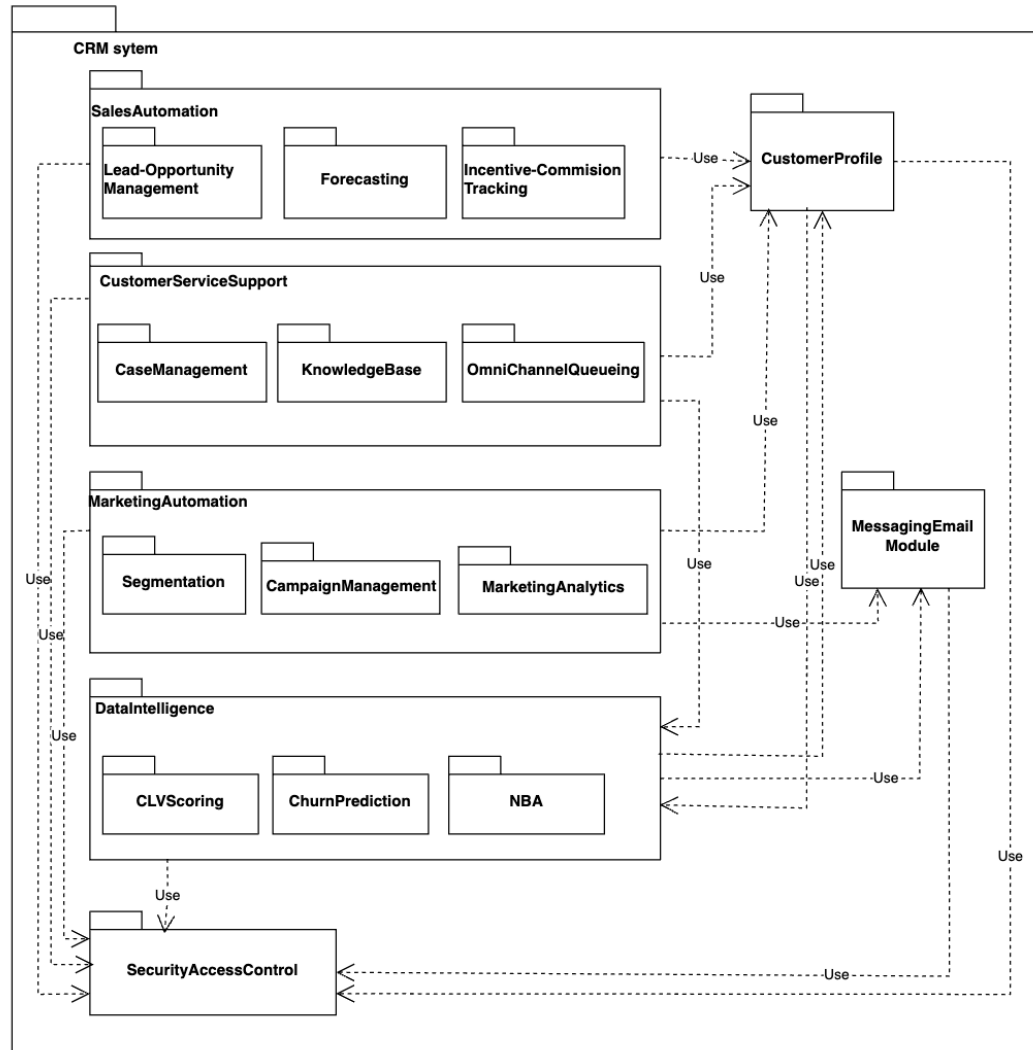
Module style focuses on the static structure of the software by showing how the system is divided into meaningful implementation units and how these units relate to each other. The goal is to show how the system is broken down into modules and how responsibility is distributed across the architecture.

#### 1.3.2.1. Decomposition & uses style

The decomposition part of the diagram shows how the CRM is partitioned into domain modules and how they are divided inside. For example, sales automation contains lead management, Forecasting and incentive tracking. This hierarchy displays the responsibilities of modules and ensures that functional requirements of the CRM is mapped to the architecture. The decomposition does not represent code packages instead, it captures how responsibilities are grouped into modules. General modules appear as separate elements because their responsibilities serve the entire system and do not belong only to a single domain.

The uses style in the diagram is represented by the dashed arrows that show how modules depend on one another. Every functional module relies on the customer profile because all CRM operations require access to customer data. Customer service support and marketing automation depend on the messaging module to send updates. Data intelligence uses messaging module when analytical results must be communicated to stakeholders. All

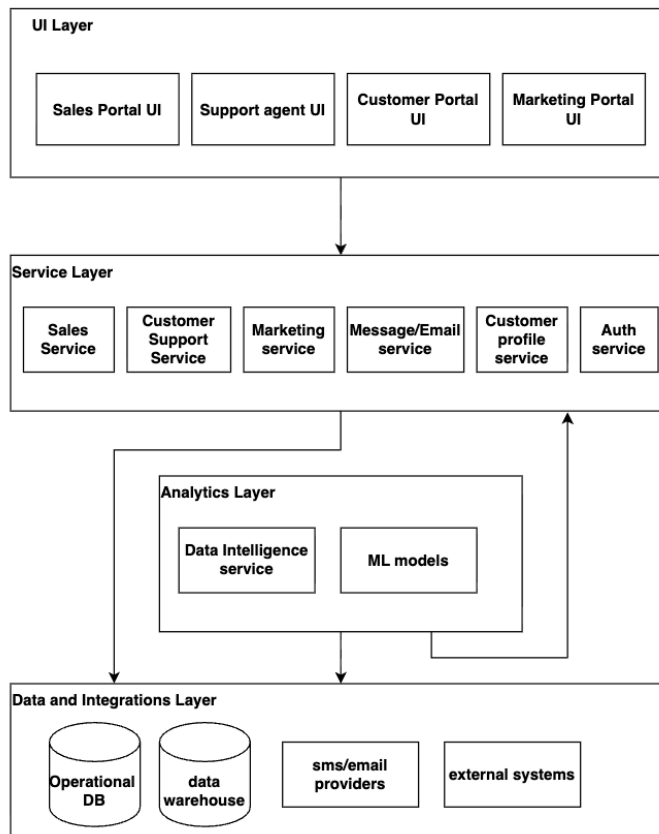
modules depend on security control because access permissions and identity checks are applied across the entire CRM platform. Sales automation and customer support use the data intelligence module to receive CLV, churn scores and next best action suggestions. These dependencies indicate that a module cannot function correctly without the services or information provided by the modules it depends on.



**Figure 2- Module style, decomposition and uses style combined diagram**

### 1.3.2.2. Layered Style

The diagram shows how the CRM system is organized into tiers. Each layer has a clear responsibility and the arrows between layers show the allowed direction of dependency. Higher layers depend on the services provided by the layers below them. The UI layer handles user interactions, displays information and sends requests to the business services. It includes no business logic. All actions performed here are forwarded to the service layer. The service layer is the core business logic of the CRM. It processes all rules and workflows required by the CRM. The UI layer depends on this layer and it acts as the main point that controls communication.



**Figure 3- Layered Style**

The analytics layer contains components responsible for decision making and prediction. This layer is separate from regular business logic because its operations involve data processing and prediction tasks. The insights it generates such as churn prediction or customer lifetime values are returned to the service layer. Finally, the data and integrations layer contains all permanent data stores and external system connections. This includes the operational database used for real time CRM transactions, the data warehouse used for historical data and external integrations. Both the service layer and the analytics layer depend on this layer. It is placed at the bottom because all upper layers rely on it for data storage, retrieval and external communication.

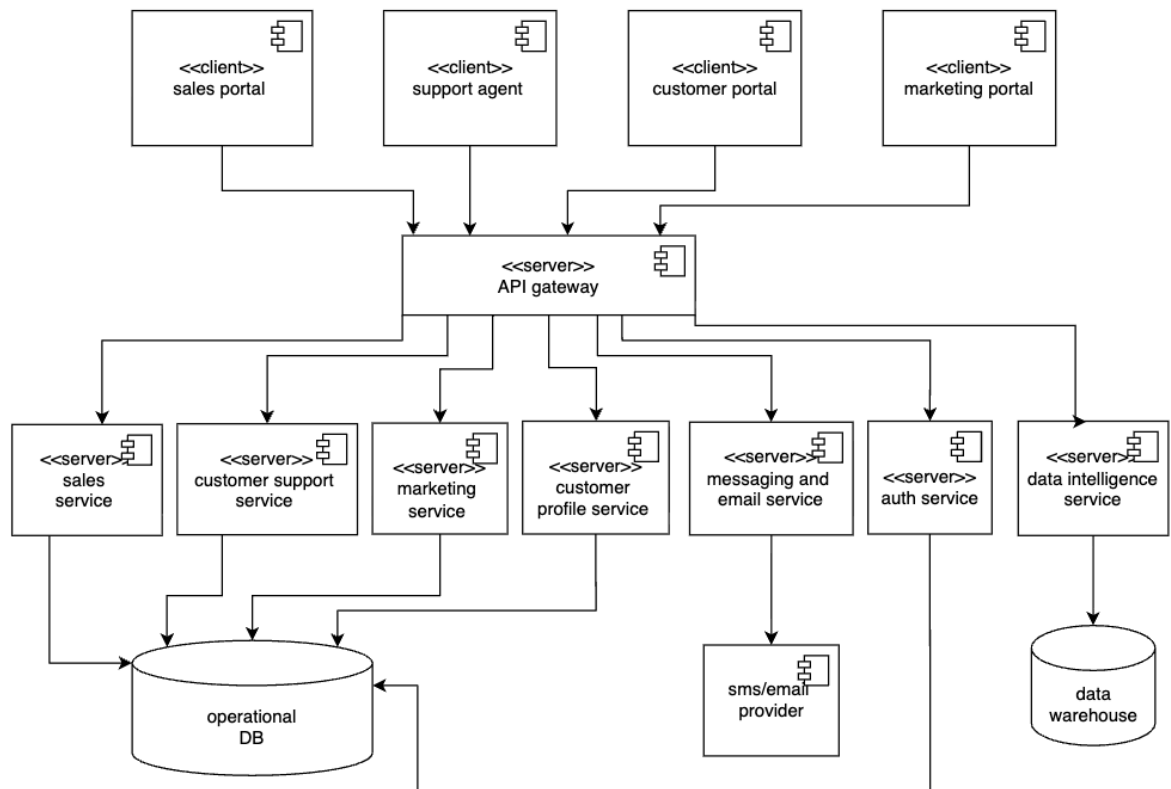
Overall, this view shows the flow of responsibility. All user interactions occur at the top, core logic is handled in the middle and all data is stored or retrieved at the bottom. This structure ensures the CRM system remains scalable, maintainable and easy to evolve.

### **1.3.3. Component & Connector Styles**

#### **1.3.3.1. Client and server diagram**

All user facing portals act as clients. Instead of talking directly to backend services, each portal sends its requests to a central API gateway. The gateway is responsible for routing the requests to the correct backend

service and makes sure that access control and communication are handled consistently.



**Figure 4- Client and server diagram**

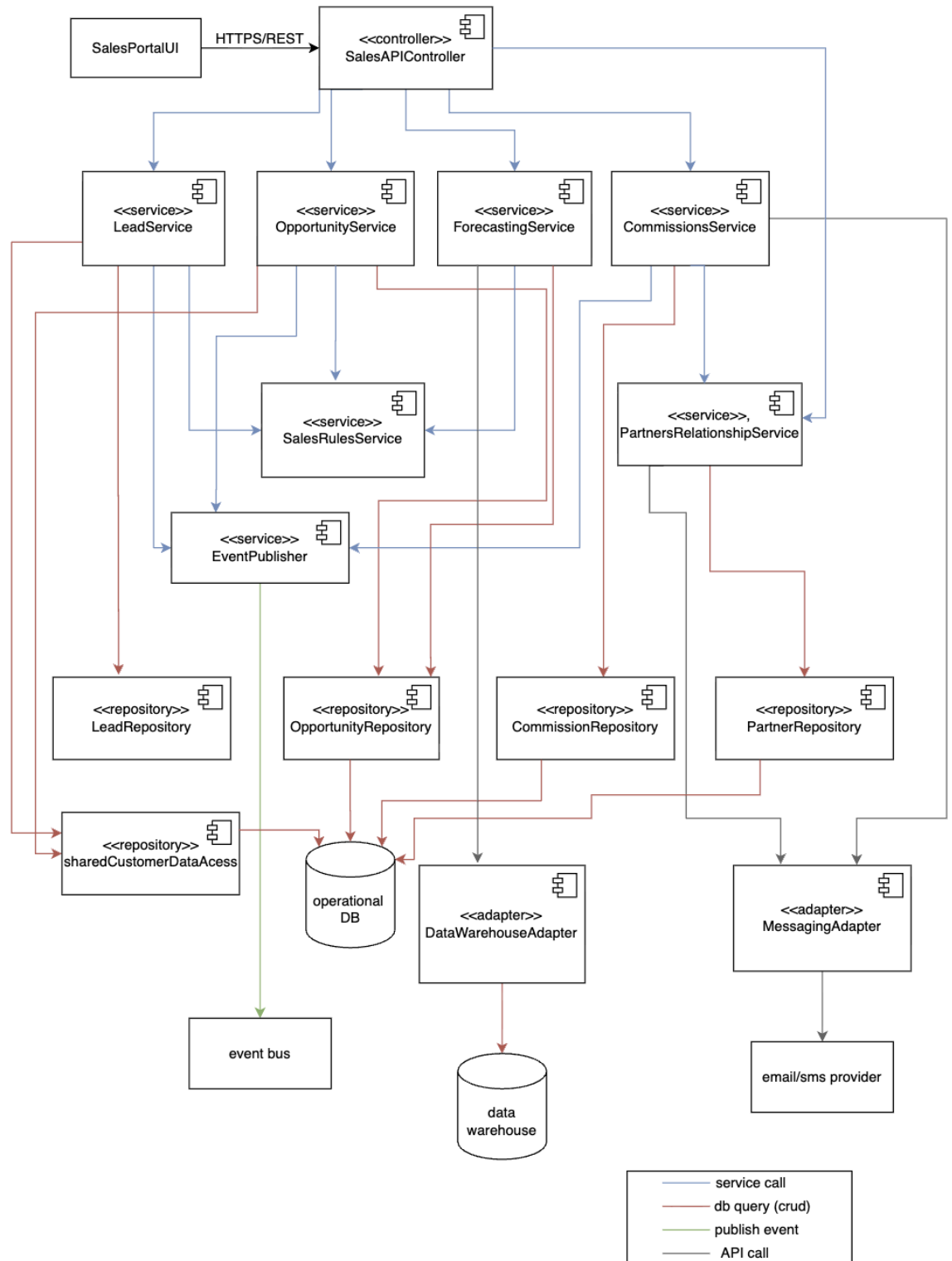
Behind the API gateway, each major CRM function is implemented as a separate backend service. The sales service manages sales operations, the customer support service handles cases, the marketing service runs campaigns, the customer profile service stores and updates customer information, the messaging and email service sends notifications through the external providers, the auth service manages login and security and the data intelligence service sends operational data into the data warehouse for reporting and analytics. Most backend services read and write their data to the shared operational db ensuring that all modules work with the same information. The external providers are only accessed through their dedicated service.

Overall, the diagram shows a system where the gateway forwards requests to independent backend services, and all services store their information in a central database. This makes the CRM easy to maintain and ensures each part of the system has a clear role.

#### **1.3.3.2. Sales automation module C&C diagram**

The component and connector view of the sales automation module shows how responsibilities are separated across controllers, domain services, repositories and how they interact through various connector types. User requests enter through UI, which communicates with the controller. The

controller orchestrates the process by delegating to domain services. These services contain business logic and coordinate with the rules service whenever rule evaluation or validation is required. Repositories save the CRM's data into the operational db. That same database is also used to send data into the data warehouse. The forecasting service uses the data in the warehouse to make better predictions.



**Figure 5- C&C diagram of sales automation module**

Asynchronous communication is modeled by showing that domain services publish domain events through the event publisher which forwards them to the event bus. This allows other CRM modules or background workflows to react to changes without tight coupling. Interactions with external systems use adapter components. Finally, shared customer data access provides a lookup mechanism for customer information that is reused by multiple domain services without duplicating data.

### 1.3.3.3. Customer Service and Support Module C&C diagram

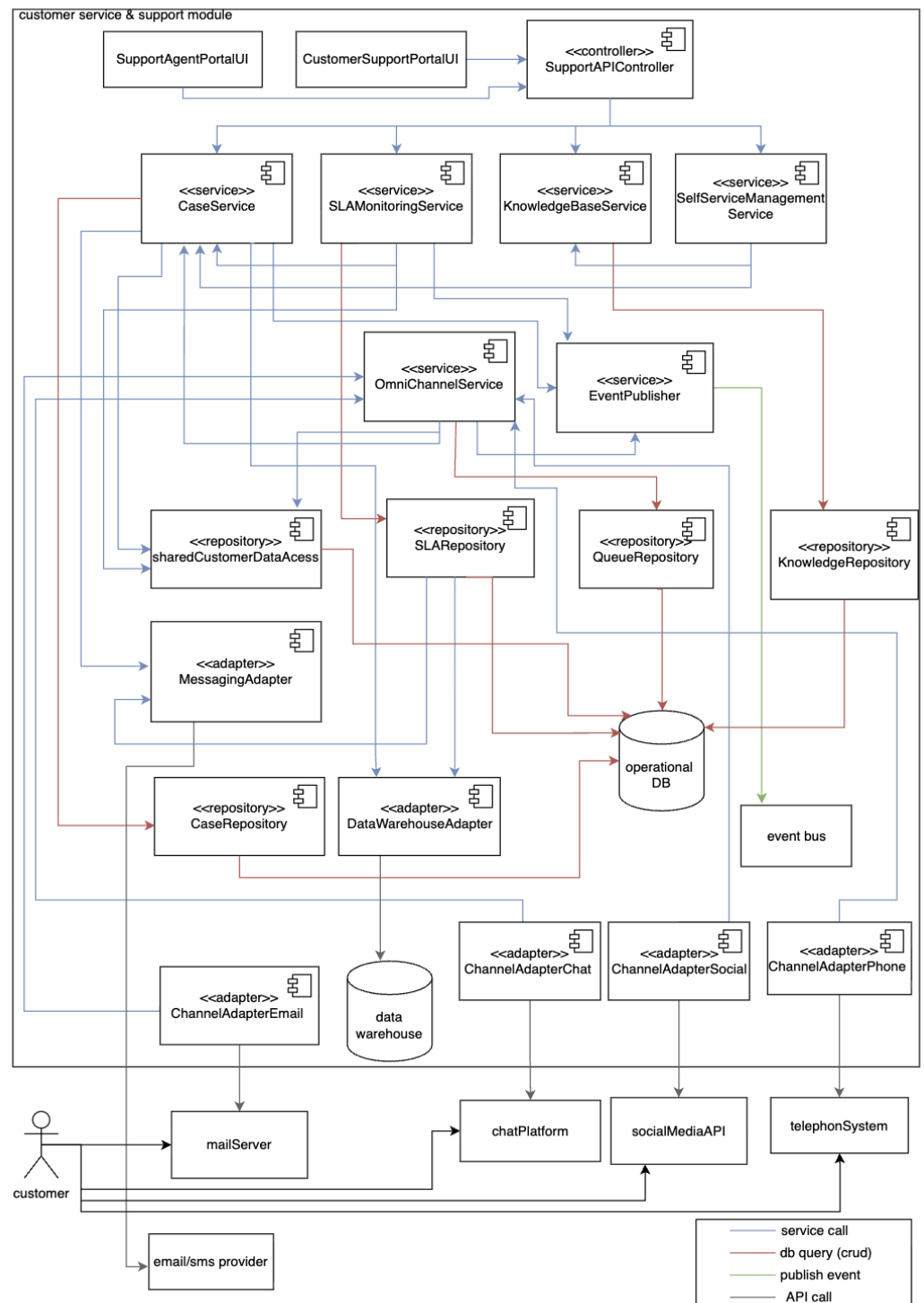
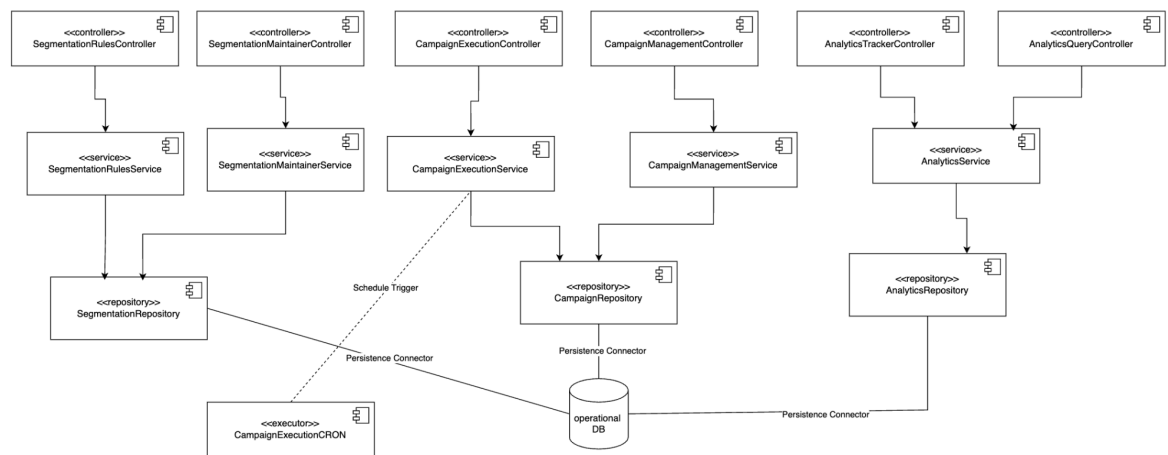


Figure 6- C&C diagram of customer support module

The customer support module follows a layered structure. At the top of the diagram, the UI components communicate with the system only through the controller. The controller acts as the single entry point and forwards each incoming request to the appropriate backend service. The core logic of the module is handled by four main services. They store and retrieve data through their own repositories. Shared customer data access is used whenever the services need customer information. Incoming messages are processed through adapters. Each adapter connects to a different external platform. These adapters receive raw customer messages, transform them into the CRM's internal format and pass them to the service. It acts as the routing point for interactions ensuring that all channels behave consistently.

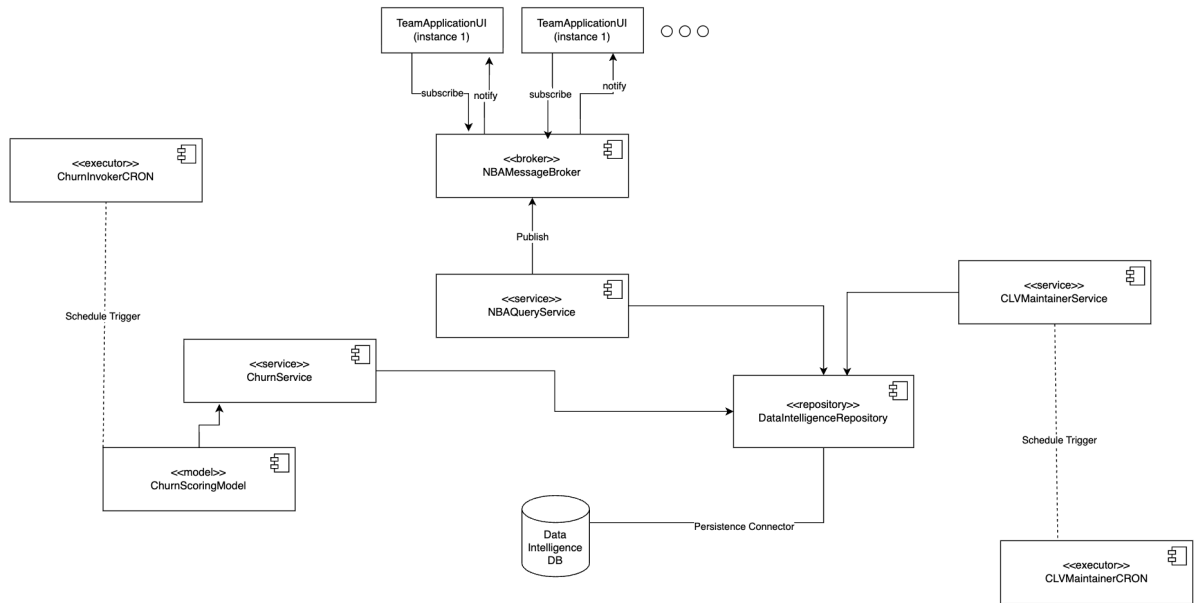
When the system needs to send notifications domain services call the messaging adapter which communicates with the external provider. This keeps all notification logic in a single place and avoids scattering external integrations across services. For asynchronous or cross module processing, services publish domain events through the event publisher. It sends these events to the event bus so that other modules or background processes can react without coupling. Analytical data is handled by the data warehouse adapter which transfers selected operational data into the data warehouse. This organization makes the customer support module easy to understand, modular and aligned with the architecture defined for the rest of the CRM.

#### 1.3.4. Marketing Automation Module C&C Diagram

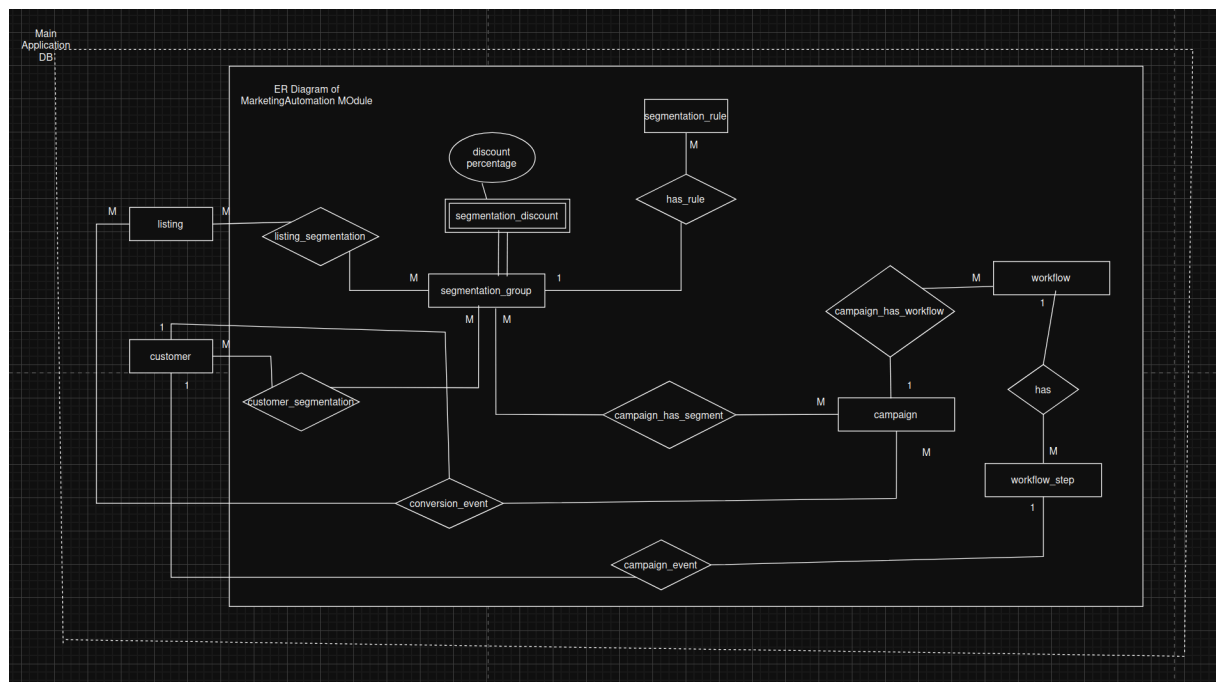


**Figure 7- C&C diagram of marketing automation module**

#### 1.3.5. Data Intelligence and Customer Insight C&C Diagram



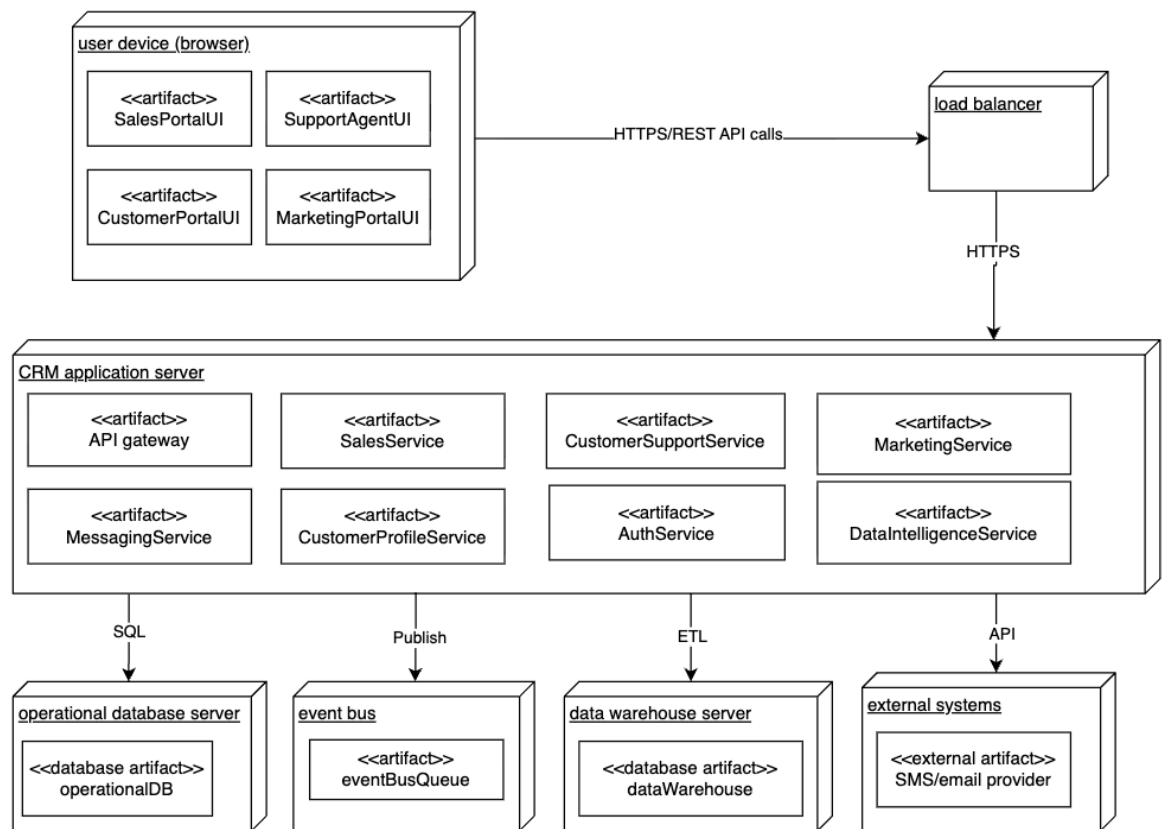
**Figure 8- C&C diagram of Data Intelligence & Customer Insight module**  
**1.3.6. ER Diagram of Marketing Automation Module**



**Figure 9- ER diagram of marketing automation module**

Figure 9 depicts the ER diagram for the Marketing Automation Module. ER diagram design for this project followed clear separation of entities and focusing on relationships between entities. Also consistency has been assured through the ER diagram and database schema.

### 1.3.7. Allocation Style- Deployment Diagram



**Figure 10- CRM system Deployment diagram**

The deployment diagram shows how the CRM system is executed across its runtime environment. All user portals run in the browser on user devices and communicate with the system through HTTPS requests routed via a load balancer to the CRM application server. This server hosts the API gateway and all backend services responsible for business logic, messaging, authentication and data processing. The application server interacts with the operational db server with SQL for transactional data, publishes asynchronous events to the event bus and performs ETL operations to populate the data warehouse server. External communication is handled through API calls. This structure separates client access, application logic, data storage, analytics and external integrations.

#### **1.4. Rationale Behind This Architecture**

The architecture is shaped directly by the stakeholder expectations, functional requirements and quality attributes identified. Because the system must serve many different teams, the architecture prioritizes modularity, shared customer data and scalability. The chosen structure ensures that each team's workflows are supported without creating tight coupling or duplicated functionality across modules.

The architecture has a modular decomposition because the system supports four major business domains. These domains represent distinct business processes each requiring

independent development, maintenance and evolution. Organizing the system as modular units allows each capability to evolve without impacting others. Also, shared modules like CustomerProfile, Messaging/Email, and SecurityAccessControl are placed outside domain layers to avoid duplication of customer data logic, communication logic and access control logic across modules. This satisfies the requirement for maintainability and consistent data usage across the entire CRM.

Furthermore, the dependency relationships among modules shown in the uses-style diagram reflect real business workflows. All functional modules rely on CustomerProfile because all CRM actions revolve around accurate customer information. SalesAutomation and CustomerServiceSupport depend on DataIntelligence to obtain churn scores, CLV values and next best action recommendations aligning with the need for intelligent decision support. Marketing automation and customer service support depend on messaging/email module to deliver campaigns and notifications across channels. These dependencies show how shared components reduce redundancy and allow centralization of critical capabilities, satisfying performance, reliability and data integrity expectations.

The adoption of a layered architectural style provides clear separation of responsibilities and supports scalability. The UI layer focuses only on interaction, ensuring usability for diverse users. The service layer contains business rules and ensures all modules follow consistent logic. The analytics layer isolates machine learning tasks making it easier to improve models independently of business workflows. The data and integration layer contains operational databases, the data warehouse and third-party providers, ensuring that all upper layers access data in a controlled way. This approach is needed for meeting nonfunctional requirements such as maintainability, performance, security, interoperability and scalability.

Moreover, user-facing interfaces communicate with backend services through a shared API gateway. This design simplifies authentication, monitoring, load balancing and access control. It also ensures that each domain service can scale independently depending on usage patterns ensuring high availability especially during heavy campaign periods. Finally, the architecture is intentionally designed to support future growth. As businesses expand, they introduce new products, flows, customer segments and marketing channels. The modular and layered structure allows new services to be added without restructuring the existing system. This ensures extensibility.

In conclusion, the architecture is justified by the need to balance independence and collaboration across teams, maintain consistent customer data, support intelligent analytics, ensure secure operations and remain scalable as the organization grows. Each architectural decision directly supports the system's functional and quality requirements.

## **1.5. Architectural patterns**

The CRM system combines several software architecture patterns. The most obvious pattern is the layered architecture pattern. The system is organized into a UI, service, analytics and data & integrations layers. Each layer has a distinct responsibility and higher layers rely on lower layers. This pattern makes the system easy to understand, maintain and modify by ensuring that user interface, business logic, analytics and data management are separated.

The system also follows client& server architectural pattern. Multiple independent client applications communicate with a centralized backend. This backend is accessed through an API gateway and performs all core processing and data operations. The client&server structure is suitable for a CRM system because it ensures that all critical logic remains on the server side where it can be secured and monitored without requiring changes to client interfaces.

Another pattern in the architecture is the service based architecture pattern. Instead of one monolithic backend, the CRM backend is composed of several domain specific services. These services operate as components inside the backend and expose functionality through defined interfaces. They are not fully independent microservices but they are clearly separated by business responsibility. This approach allows each major capability of the CRM to evolve independently while still benefiting from shared infrastructure and shared data.

Finally, the system follows a domain oriented architecture. Rather than organizing components around technical layers, it mirrors the real structure of the business by grouping functionality into domains such as sales automation, customer service, marketing automation, and data intelligence. Each domain contains its own submodules and workflows while elements that are used by all modules like customer profile and messaging module are treated as shared components supporting the entire platform. This pattern ensures that the architecture aligns with how different teams operate and makes the system easier to scale and extend.

Overall, the architecture combines layered architecture, client& server interactions and domain oriented structuring. Together they create a platform that is aligned with business needs and suitable for a CRM system that must support multiple departments, diverse users and operational, analytical workloads.

## 2. Implementation



The name of our product is CMS meaning “CRM Marketing System”.

## 2.1. High Level Implementations

### 2.1.1. Dashboard

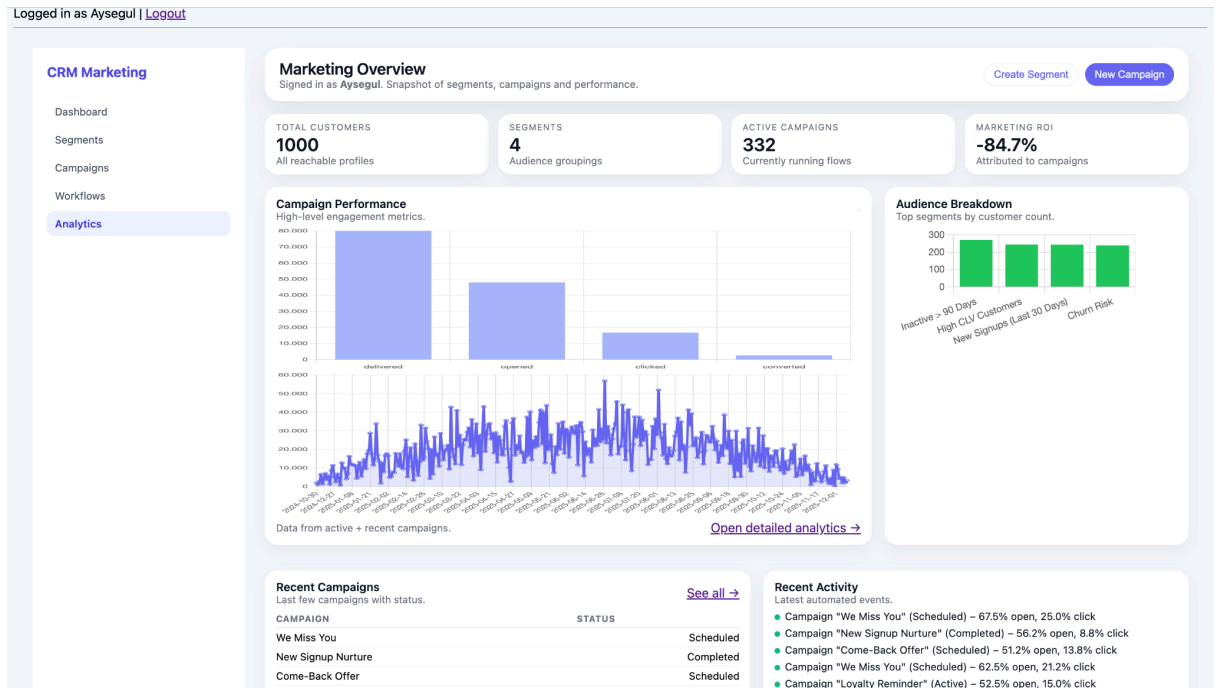


Figure 11- Dashboard

The dashboard page is the main entry point of the CRM marketing system and gives users a quick overview of what is happening in their marketing environment. As soon as the user logs in, the page shows four key numbers which are the total number of customers in the CRM, audience segments, campaigns that are currently active and the overall marketing ROI. These values help the user understand the general performance of the system.

Below the KPIs, the dashboard displays two important charts about campaign performance. The conversion graph shows how many campaign messages were delivered, opened, clicked and converted. This helps the marketing team see where engagement drops. Under it, the revenue over time chart shows how much revenue was generated each day from conversions. The user can quickly notice trends or patterns in campaign performance by looking at this graph.

On the right side, the dashboard includes an audience breakdown chart showing how customers are distributed across segments. It helps the user to understand the structure of their customer base. Recent campaigns are listed below together with their statuses so the user can see what has been happening in the system. A recent activity feed also shows the latest automated events.

All of the information on the dashboard is generated automatically by reading the mock data warehouse in the project. The system processes customer records, campaign events and conversion events and turns them into the metrics and charts seen on the page.

## 2.1.2. Campaigns Page

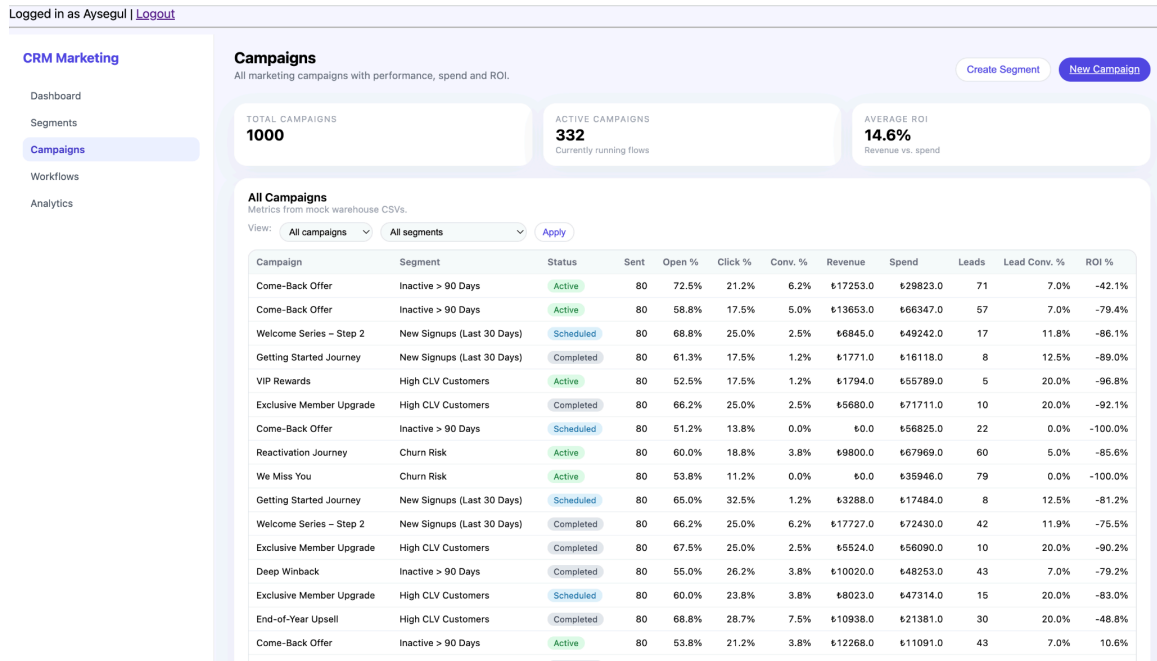


Figure 12- campaigns page

The campaigns page gives users a complete view of all marketing campaigns in the CRM system and allows them to understand how each campaign is performing. At the top of the page, the total number of campaigns in the system, currently active campaign number and the average ROI across all campaigns are shown. They help users see the overall scale of their marketing efforts.

The main feature of this page is the campaign table, which lists every campaign along with its most important performance metrics. Each row shows the campaign's name, the customer segment it targets, its current status and key engagement numbers such as how many messages were sent, open rate, click rate, conversion rate and total revenue generated. The table also includes the number of leads generated by the campaign and the lead conversion percentage. Together, they show how each campaign is performing. To help users explore the data, the page includes simple filter controls. Users can filter campaigns by status or by audience segment. When they apply a filter, the table shows only the campaigns relevant to their selection. This allows them to focus on specific workflows. Event data is used to compute metrics like open and click rates, while conversion data is used to calculate revenue. This makes the campaigns page a detailed but easy to understand reporting tool where users can compare campaigns, find underperforming ones, and understand which strategies deliver the highest return.

## 2.1.3. Segmentation Groups, Workflows, Campaigns & Automated

The system allows creating segmentation groups to organize users and listings, either manually or through rules, so that each group contains relevant entities. Staff users can then manage campaigns and assign multiple workflows to a campaign, with each workflow belonging to exactly one campaign (1-to-many

relationship). Each workflow consists of ordered workflow steps that can perform actions like sending emails, posting to Discord, or assigning discounts. We implemented the logic which runs these steps sequentially, supports delays between steps, and can handle failures, all. A multithreaded execution script for workflow execution is provided, which can be run through a cron job. The system also provides monitoring and control, so staff can track the progress and status of each workflow step. Also, we track events in the campaign\_eventable, recording actions taken for each customer, workflow step, and campaign.

Logged in as Aysegul | [Logout](#)

**CRM Marketing**  
  
Dashboard  
Segments  
**Campaigns**  
Workflows  
Analytics  
  
Manage your campaigns.  
[View analytics](#)

### Create New Campaign

Set up a new campaign and manage existing ones in a single view.

#### New Campaign

Define the name and initial status of your campaign.

Campaign Name

Status Planned

[Create Campaign](#)

#### Existing Campaigns

Filter by status and edit an existing campaign.

Filter by status: All

No campaigns found.

[Edit Selected Campaign](#)

Figure 13- create campaigns page

Logged in as yigit | [Logout](#)

### Create Segmentation Group

Group Name:  [Create Group](#)

**Bilkent Alumni**  
ID: 2

**young-adult**  
ID: 1

[Edit Selected Group](#)

**Users**  
Yigit Yayla —  
yigit.yayla@ug.bilkent.edu.tr (ID: 1)

**Listings**  
headphones — \$300.00 (ID: 1)

[Add Selected User](#) [Add Selected Listing](#)

Figure 13- create segmentations page

Logged in as yigit | [Logout](#)

Workflow Editor

workflow2 (ID: 2)

Here you will define workflow steps.

Add New Step

Step Order:

Action Type:

Discord Post

Action Payload (JSON or Text):

Payload content...

Delay Minutes After Previous Step:

0

Add Step

Step : EMAIL

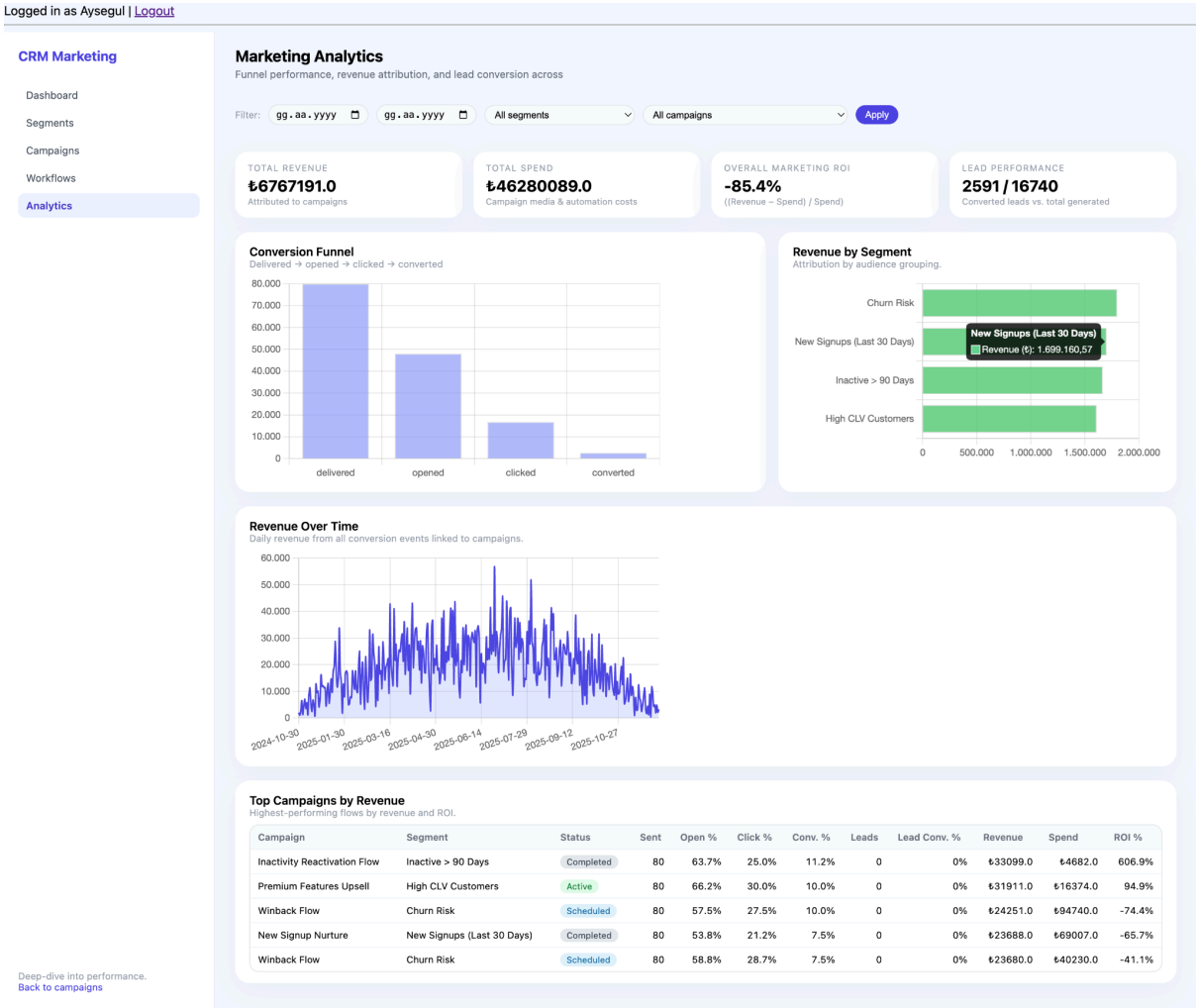
ID: 1

Hi, this is a test email from CRM system.

Delay: 0 minutes

Figure 14- workflow editor

2.1.4. Analytics Page



The analytics page shows how well campaigns are performing. While the dashboard and campaigns pages give a general overview, the analytics page provides a deeper and more detailed breakdown by combining user conversion metrics, revenue attribution, segment impact, and time based trends. The page starts with a filter section that allows the user to narrow the results by date, audience segment and campaign. These filters update every chart and metric on the page. This flexible filtering is important for measuring campaign effectiveness more accurately and answering questions like “Which segment brought the most revenue this month?” or “How did our new signup flow perform last month?”.

The top summary cards display the most important KPIs which are total revenue attributed to campaigns, total marketing spend, the overall ROI calculated from these numbers and the lead conversion count. These values help the user quickly understand whether marketing activities are generating profit or loss within the selected time frame. The conversion funnel chart visualizes how leads move through the stages allowing the user to see where most drop-offs happen and how efficiently the campaign drives conversions. The revenue by segment chart shows which audience groups contribute the most value. This makes it easy to compare segment behavior and identify which groups respond better to marketing efforts.

The revenue over time chart displays daily revenue generated by all recorded conversion events linked to campaigns. This view helps the user identify trends, and declines in performance. It also helps evaluate whether certain periods or seasonal factors affect revenue. Finally, the page includes a top campaigns by revenue table which highlights the highest performing campaigns and lists all key metrics. This table allows quick comparison between campaigns and helps identify which strategies are worth repeating or scaling. Overall, this page provides detailed measurement and insight into segmentation effectiveness, funnel performance, revenue attribution and ROI.

## **2.2. Key Implementation Snippets (Code-Level)**

### **2.2.1. Project structure**

The project structure reflects a layered and modular architecture. Each responsibility is grouped into a dedicated module such as routes, services, repositories, models and templates. This separation ensures that presentation logic, business logic, data access and database models remain independent from each other. For example, route files only handle HTTP requests and delegate the core logic to service classes while repositories interact with the underlying data sources. In our implementation, the app/data directory contains CSV files that function as a simplified data warehouse for the entire CRM. In a real world company, these datasets would come from large relational databases or enterprise data lakes, but for this project the csv-based warehouse allows us to simulate customer records, campaign events, segmentation information, and

conversion histories. All analytics and marketing automation features read from these files just as they would query a real warehouse.

Project structure:

```
app/
├── auth/
│   ├── decorators.py
│   └── routes.py
├── marketing/
│   └── routes.py
├── campaign/
│   └── routes.py
├── services/
│   ├── analytics_service.py
│   ├── CampaignExecutionService.py
│   ├── CampaignManagementService.py
│   └── ...
├── repositories/
│   ├── CampaignRepository.py
│   └── ....
├── models/
│   └── .....
├── templates/
│   ├── marketing/
│   ├── campaign/
│   └── base.html
├── data/
│   └── .....
├── database.py
├── config.py
└── run.py
```

### 2.2.2. Multithreaded CRON Scripts

In order to periodically execute workflows we implemented a multithreaded python script, which can be invoked by a CRON job. With this script workflow steps to be executed are divided into chunks and each thread executes steps and if an exception occurs or if it is done persists it in the database. By parallel computing we gain high performance. And since Python is synchronous by default, we can import an asynchronous execution library to attain further performance as future work.

```
import threading
import time
from typing import List
from app.models.WorkflowStepModel import WorkflowStepModel
from app.services.CampaignManagementService import
CampaignManagementService
```

```

# --- Worker thread ---
def worker(steps: List[WorkflowStepModel]):
    for step in steps:
        try:
            # Execute step
            CampaignExecutionService.execute_step(step)
        except Exception as e:
            step['status'] = 'FAILED'
            print(f"Step {step['id']} failed: {e}")

# --- Main execution ---
def run_workflow_steps_concurrently(all_steps: List[dict],
num_threads=4):
    if not all_steps:
        print("No workflow steps to execute.")
        return

    # Split steps into roughly equal chunks for each thread
    chunks = [[] for _ in range(num_threads)]
    for i, step in enumerate(all_steps):
        chunks[i % num_threads].append(step)

    threads = []
    for chunk in chunks:
        if chunk: # Only start thread if there are steps
            t = threading.Thread(target=worker, args=(chunk,))
            t.start()
            threads.append(t)

    # Wait for all threads to complete
    for t in threads:
        t.join()

    print("All threads finished execution.")

```

### 2.3. Tests based on the requirements

We tested the marketing automation module by validating each feature against the functional requirements and confirming that the system behaves correctly when working with our sample data warehouse, which is represented by csv files. For the dashboard, we first checked that key totals such as number of customers, segments, active campaigns, total spend, total revenue and ROI match the real values computed

from the csvs. This ensured that the summary metrics update automatically and accurately.

On the campaigns page, we tested the filtering system by selecting different statuses and segment options, and checked that the list updates correctly. We also manually recalculated each campaign metric (open rate, click rate, conversion rate, revenue, spend and ROI) from the event and conversion data to confirm that the values shown in the interface match the underlying data. This validates correct implementation of both data aggregation and filtering on the page.

For the analytics page, we tested the behavior of date, segment and campaign filters. When choosing a time range, we checked that the KPIs and all charts only include events and conversions inside that window. We confirmed that the conversion funnel counts delivered, opened, clicked and converted events correctly. We verified that the revenue over time graph displays the correct daily totals, and that revenue by segment aligns with the sum of revenues generated by campaigns belonging to that segment. We also applied filters that produce no matching data to ensure the system still returns valid empty results.

In addition to manual testing, we added automated tests using pytest. These include verifying that the dashboard, campaigns page and analytics page load successfully confirming that campaign filtering works, checking that analytics filters change the returned content and ensuring that the revenue-over-time chart element appears in the rendered HTML. Login protected routes were tested using a logged in client fixture to simulate an authenticated session. These automated tests help verify that the application behaves correctly after changes and reduce the chance of regressions.

We also evaluated non functional requirements. Because all pages require login, the system satisfies basic security expectations. The application responds quickly with realistic dataset sizes, supporting performance needs. The interface uses clear labels, tables and charts, which improves usability. The data pipeline loads everything from CSV files, but the logic is written so that switching to a real database does not require major changes. This supports scalability and maintainability. Altogether, the tests confirm that the marketing automation module works correctly for designing campaigns, tracking performance and analyzing conversions, and that it can be extended to larger and more realistic environments.

#### **2.4. Comparison with Existing Marketing Automation Tools**

Our CRM Marketing System is a smaller and simpler version of the tools used in the real world. When we compare it with known platforms like Insider One, Salesforce Marketing Cloud, Adobe Experience Cloud, Oracle Marketing Cloud, and HubSpot Marketing Hub, the main differences come from scale, automation power and the

variety of features. Even though these platforms are much larger, the basic ideas behind the system are the same.

Just like professional systems, our module starts with customer segmentation. We support both manual and rule based segmentation so that customers and listings can be grouped into meaningful segments. Big platforms use AI and real time data to build these groups automatically but the goal is the same which is to create targeted and personalized marketing for each customer segment.

Our campaign and workflow system also follows the logic used by industry platforms. In our system, a campaign is a high level marketing activity and each campaign can contain multiple workflows. Each workflow is made up of ordered steps such as sending an email, posting on social media or applying a discount. A cron job runs these steps over time following delays when defined. This is a simpler version of the “customer journeys” found in Salesforce, Adobe, Oracle, HubSpot, or Insider. Those platforms offer much more such as branching paths, personalized actions and many channel integrations. Still, our implementation correctly models the core idea of automated multi step marketing.

Our analytics also mirror what major platforms do, but at a smaller scale. We calculate conversions, revenue, spend, ROI and show graphs for revenue over time, segment performance, and conversion stages. Real enterprise tools provide deeper insights, including predictive analytics and advanced attribution models but the purpose is the same which is helping marketers understand how their campaigns perform and how customers respond. Another difference is in the data infrastructure. Real systems rely on large databases, real time event streams and cloud storage. In our project, we use CSV files as the company’s “data warehouse.” While this is a simplified approach, it still represents the idea of having a single data source that segmentation, campaigns and analytics use.

In conclusion, while the major platforms offer far more features, automation, and real-time capabilities, our system successfully captures the core functions of marketing automation. It provides segmentation, campaign execution, workflow automation and performance reporting in a way that is realistic for an academic project and aligned with how professional CRM systems operate.